

02 Global Songs data extraction from Spotify

Data Engineering Project: Serverless Spotify ETL Pipeline - Extraction & Transformation Fundamentals

This section will cover the initial steps of our project: **extracting data from the Spotify API** and **transforming it into a structured format** suitable for analysis. We'll be using Python for this entire process.

1. Setting Up Spotify API Access

To interact with Spotify's data, we need to register our application and obtain credentials.

- **Access the Spotify Developer Dashboard:**
 - Go to <https://developer.spotify.com/dashboard>
 - **Log in or Sign Up** for a Spotify account. You can use your phone number, Google, Apple, or Facebook for sign-up.
- **Create an Application:**
 - Once logged in, click on **"Create App"**.
 - Give your application a name (e.g., "Spotify ETL Pipeline").
 - Agree to the terms and click "Create".
- **Obtain Client ID and Secret ID:**
 - Upon creating the app, you will receive a **Client ID**.
 - Click to reveal and copy your **Secret ID**.
 - **Important Security Note:** Your Secret ID should **never be shared publicly**. You would protect this key (e.g., using environment variables or a secrets manager).

2. Python Setup and Spotify API Authentication

We'll use a specific Python package to simplify our interaction with the Spotify API.

- **Install spotipy :**
 - Open your terminal or command prompt and run: `pip install spotipy`. This package provides a convenient way to access Spotify API functions.
- **Import Necessary Modules:**
 - In your Python script, import `spotipy` and `spotipy.oauth2.SpotifyClientCredentials`.

```
import spotipy
from spotipy.oauth2 import SpotifyClientCredentials
```

- **Authentication:**
 - Provide your **Client ID** and **Secret ID** to the `SpotifyClientCredentials`. This step is crucial for **authentication**, granting you permission to extract data.

```
# Replace with your actual Client ID and Secret ID
client_id = "YOUR_CLIENT_ID"
```

```
secret_id = "YOUR_SECRET_ID"

# Authenticate with Spotify API
sp = spotipy.Spotify(auth_manager=SpotifyClientCredentials(client_id=client_id,
secret_client=secret_id))
```

– The `sp` object (created by `spotipy.Spotify`) is your main interface for interacting with the Spotify API.

3. Data Extraction - Top Global Songs Playlist

Our goal is to extract data from the **top 100 trending songs globally on Spotify**.

- **Identify the Playlist URL:**
 - Locate the specific URL for the "Top Global Songs" playlist (example provided in source is a Spotify URL for a playlist).
 - Example: `https://open.spotify.com/playlist/37i9dQZEVXbMD0HDwVN2tF` (Example URL, actual URL may differ).
- **Extract the Playlist URI:**
 - The Spotify API functions typically expect a **Playlist URI** (a shorter identifier) rather than the full URL.
 - We will perform a **basic string transformation** to extract this:
 1. **Split by /** : This converts the URL string into a list.
 2. **Access the last element**: This isolates the part containing the playlist ID and query parameters.
 3. **Split by ?** : This separates the playlist ID from any additional query parameters.
 4. **Access the first element**: This gives you the clean Playlist URI.

```
playlist_link = "YOUR_SPOTIFY_PLAYLIST_URL" # e.g.,
"https://open.spotify.com/playlist/37i9dQZEVXbMD0HDwVN2tF?si= ..."
playlist_uri = playlist_link.split('/')[ -1 ].split('?')[ 0 ]
```

- **Fetch Playlist Tracks:**
 - Use the `playlist_tracks` function from the `sp` (Spotify object) and pass the `playlist_uri`.
 - This will return **JSON data** containing detailed information about the tracks in the playlist.

```
data = sp.playlist_tracks(playlist_uri)
```

– The returned `data` variable will hold this raw, often nested, JSON structure.

4. Data Transformation: Structuring the Raw JSON

The raw JSON data from Spotify API is often nested and needs to be transformed into a flat, structured format suitable for tables or dataframes. Our goal is to create separate structures for **albums, artists, and songs**.

Understanding the JSON Structure

- The primary information about songs, albums, and artists is nested under the `['items']` key in the raw data.
- Inside `['items']`, each element is typically a dictionary containing track information, which in turn holds album and artist details.
- Crucially, a single song can have **multiple artists**, meaning the artist information is often a **list of dictionaries** within the track's data. This requires careful handling with nested loops.

4.1 Extracting and Structuring Album Data

We will iterate through the `data['items']` to extract album-specific information for each song.

- **Identify Album Fields:**
 - Within `data['items'][index]['track']['album']`, you will find keys like `id`, `name`, `release_date`, `total_tracks`, and `external_urls` (which contains a spotify URL).
- **Loop and Extract:**
 - Create an empty list (e.g., `album_list`) to store extracted album dictionaries.
 - Loop through each row (which represents an item/song) in `data['items']`.
 - Inside the loop, access the album information for the current row.
 - Extract the `album_id`, `album_name`, `release_date`, `total_tracks`, and `album_url` for each album.
 - Create a **dictionary** for each album, mapping field names to their extracted values.
 - **Append** this album dictionary to `album_list`.

```
album_list = []
for row in data['items']:
    album_id = row['track']['album']['id']
    album_name = row['track']['album']['name']
    album_release_date = row['track']['album']['release_date']
    album_total_tracks = row['track']['album']['total_tracks']
    album_url = row['track']['album']['external_urls']['spotify']
    album_element = {
        'album_id': album_id, 'name': album_name, 'release_date': album_release_date,
        'total_tracks': album_total_tracks, 'url': album_url
    }
    album_list.append(album_element)
```

- This process transforms messy JSON into a structured list of dictionaries for albums, making it much more readable and usable.

4.2 Extracting and Structuring Artist Data

Artist data is more complex due to multiple artists per song, requiring nested loops.

- **Identify Artist Fields:**
 - Within `data['items'][index]['track']['artists']`, you'll find `id`, `name`, and `external_urls` (containing spotify URL).
- **Nested Loop for Artists:**
 - Create an empty list (e.g., `artist_list`) to store extracted artist dictionaries.
 - Loop through each row (item/song) in `data['items']`.

- Inside this loop, identify the track information.
- Access the artists list within that track.
- **Loop again** through each artist dictionary within the artists list.
- Extract artist_id, artist_name, and artist_url for each artist.
- Create a dictionary for each artist.
- **Append** this artist dictionary to artist_list.

```
artist_list = []
for row in data['items']:
    for key, value in row.items():
        if key == "track":
            for artist in value['artists']:
                artist_dict = {'artist_id':artist['id'],
                              'artist_name':artist['name'], 'external_url': artist['href']}
                artist_list.append(artist_dict)
```

4.3 Extracting and Structuring Song/Track Data

(The process for songs is similar to albums; it involves iterating through items and extracting relevant track-level details like id, name, duration_ms, popularity, added_at etc.).

- **Identify Song Fields:**
 - Within data['items'][index]['track'], you'll find keys like id, name, duration_ms, popularity, added_at (from the parent item dictionary), and more.
- **Loop and Extract:**
 - Similar to the album extraction, loop through data['items'].
 - Extract track-specific information.
 - Create a dictionary for each song.
 - Append to a song_list.

```
song_list = []
for row in data['items']:
    song_id = row['track']['id']
    song_name = row['track']['name']
    song_duration = row['track']['duration_ms']
    song_url = row['track']['external_urls']['spotify']
    song_popularity = row['track']['popularity']
    song_added = row['added_at']
    album_id = row['track']['album']['id']
    artist_id = row['track']['album']['artists'][0]['id']
    song_element =
{'song_id':song_id, 'song_name':song_name, 'duration_ms':song_duration, 'url':song_url,

'popularity':song_popularity, 'song_added':song_added, 'album_id':album_id,
  'artist_id':artist_id
}
    song_list.append(song_element)
```

5. Converting to Pandas DataFrames and Initial Cleaning

Once we have our lists of dictionaries (e.g., `album_list`, `artist_list`, `song_list`), we can convert them into **Pandas DataFrames** for easier manipulation and analysis.

- **Import Pandas:**

```
import pandas as pd
```

- **Create DataFrames:**

- Use `pd.DataFrame.from_dict()` to convert your lists of dictionaries into DataFrames.

```
album_df = pd.DataFrame.from_dict(album_list)
artist_df = pd.DataFrame.from_dict(artist_list)
song_df = pd.DataFrame.from_dict(song_list)
```

- **Data Type Conversion (Transformation):**

- **Release Date & Added At:** Dates are often extracted as `object` (string) types. For proper date-based operations, convert them to `datetime` objects using `pd.to_datetime()`.

```
album_df['release_date'] = pd.to_datetime(album_df['release_date'])
song_df['song_added'] = pd.to_datetime(song_df['song_added'])
```

- **Handling Duplicates:**

- It's good practice to drop duplicate records, especially for entities like albums and artists, to ensure data integrity. Songs are generally unique in this context, but albums and artists might repeat across different tracks.
- Use `df.drop_duplicates(subset=['column_name'])`.

```
album_df.drop_duplicates(subset=['album_id'], inplace=True)
artist_df.drop_duplicates(subset=['artist_id'], inplace=True)
# song_df typically won't have duplicates for this specific use case,
# but for other scenarios, you might drop duplicates on a song_id.
```

– The `'inplace=True'` argument modifies the DataFrame directly.

6. Recap of Extraction and Transformation

At this stage, we have successfully:

- **Extracted** raw, nested JSON data from the Spotify API.
- **Transformed** this messy data into three distinct, structured Pandas DataFrames: `album_df`, `artist_df`, and `song_df`.
- Performed **basic cleaning** by converting data types (e.g., to datetime) and removing duplicates.

This structured data is now ready for further analysis, such as joining tables, performing aggregations (e.g., "which album had the highest total number of tracks"), or loading into a data warehouse.

7. Next Steps: Cloud Deployment and Automation (ETL Load Phase)

The next phase of this project will focus on the **Load** part of ETL, involving the **deployment and automation** of this entire pipeline onto AWS cloud services. We will explore how to schedule this process to run weekly and store our transformed data for future querying and analysis.
